# CSC0071 Meta-heuristics and Problem Solving
# Feature-Guided Domain Selection for Fast Fractal Image Compression of High-Resolution Images

Course Instructor: Prof. Tsung-Che Chiang

Students: Zhen-Rong Wu, Darrin Lin, Liang-Yun Ko

Department of Computer Science and Information Engineering
National Taiwan Normal University

June 2026

# Outline

1. **Background**: what is Fractal Image Compression (FIC), and why is it slow?
2. **Motivation**: prior work uses PSO — but it breaks at high resolution
3. **The path to our method**: PSO → Memetic PSO → FG-PSO → the key ablation
4. **FGDS**: feature extraction, KD-Tree candidate selection, exhaustive search
5. **Genetic Algorithm**: learning which features matter
6. **Experiments**: four connected experiments and what they prove
7. **Conclusion** and contributions

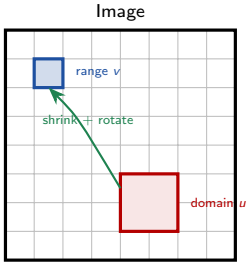
## One-sentence summary

We move the heuristic from **block matching** to **block selection**: instead of using a metaheuristic to *match* blocks, we use features to *select* a small, high-quality candidate pool, then search it exhaustively.

# What is Fractal Image Compression (FIC)?

**Core idea: an image is similar to itself at different scales.**

- A small range block often looks like a shrunk, rotated copy of a larger region elsewhere in the image (a domain block).
- Instead of storing pixels, we store *instructions*: "this block = that region, shrunk, rotated, with contrast $s$ and brightness $o$."
- These instructions form a set of *contractive affine maps* (an Iterated Function System).




$$\hat{v} = s \cdot T_k(\text{downsample}(u)) + o$$

$T_k$: one of 8 rotations/flips; $s$: contrast; $o$: brightness.

## Why it is attractive

- High compression ratio.
- **Resolution-independent** decoding: zoom in without blocking artifacts.

# Encoding vs. decoding: the asymmetry

## Decoding is fast

Start from *any* image, repeatedly apply the stored maps. By Banach's fixed-point theorem it converges to a unique reconstruction in ∼20 iterations.

## Encoding is slow

For **every** range block we must find the **best** domain block and isometry:

$$\min_{u \in \mathcal{D},\ k \in \{0..7\}} \mathrm{MSE}\big(v,\ s\, T_k(u) + o\big)$$

$s, o$ have a closed form; the hard part is the *search* over $u$ and $k$.

**The cost explodes with resolution.**

- For a $1024 \times 1024$ image with $4 \times 4$ range blocks:

  $M = 65{,}536$ range blocks
  $P = 65{,}025$ domain blocks
  $\times\, 8$ isometries

- Full search $= M \times P \times 8 \approx \mathbf{3.4 \times 10^{10}}$ evaluations *per image*.

## The research question

Can we keep the quality of full search while drastically cutting this cost?

# One "fitness evaluation" (FFE) — the unit of cost

For a fixed range block $v$, domain block $u$, and isometry $k$, the optimal contrast and brightness have a **closed-form least-squares solution**:

$$s = \frac{\mathrm{Cov}(v, u_k)}{\mathrm{Var}(u_k)}, \qquad o = \bar{v} - s\,\bar{u}_k,$$

$$\mathrm{MSE} = \tfrac{1}{L^2} \textstyle\sum (v - s\,u_k - o)^2.$$

- One such computation = one Fitness Function Evaluation (FFE).
- **Every** method in this talk is measured in FFEs — this makes the comparison fair.
- Contractivity $|s| < 1$ is required for the decoder to converge.

## Key observation we will exploit

- A **mean** mismatch ($\bar{v} \neq \bar{u}$) is absorbed *exactly* by the offset $o$ — it costs nothing.
- A **contrast** mismatch ($\sigma_v \neq \sigma_u$) cannot be absorbed — it directly inflates the MSE.

$\Rightarrow$ Standard deviation matters far more than mean for choosing a good domain. (We will let a GA discover this.)

# Prior work: replace the search with a metaheuristic

**The standard approach** (Muruganandham & Wahida Banu, 2010): treat $(d, k)$ — domain index and isometry — as the variables, and search with Particle Swarm Optimization (PSO).

- Each particle = a candidate $(d, k)$.
- Particles fly through the search space, pulled toward their own best and the swarm's global best.

$$\mathbf{V} \leftarrow w\mathbf{V} + c_1 r_1(\mathbf{P}_{best} - \mathbf{X}) + c_2 r_2(\mathbf{G}_{best} - \mathbf{X})$$

Genetic algorithms and pyramid-PSO have also been tried in the same spirit.

## The problem at high resolution

- One range block has $P \times 8 \approx \mathbf{5.2 \times 10^5}$ candidates.
- A swarm of 40 particles covers $< 0.01\%$ of that at start.
- The objective over domain indices is *not smooth*: neighboring domains can be totally different.
- $\Rightarrow$ the swarm has no gradient to follow and gets stuck far from the optimum.

**Our diagnosis:** the weakness is not the optimizer — it is the **quality of the search space** we hand it.

# Our reasoning trail (each step is an experiment)


## The decisive insight

Once the candidate pool is small (a few hundred), **exhaustive search beats PSO** inside it — it is exact, deterministic, and uses fewer evaluations. So we **remove the metaheuristic matcher** and keep only the feature-guided *selection*. That selection becomes our contribution.

# The three baselines, briefly

## 1. Standard PSO

A swarm of 40 particles searches the full $(d, k)$ space. Cheap per step, but lost in a huge space.

## 2. Memetic PSO (= PSO + local search)

Every few iterations, refine the best particles:

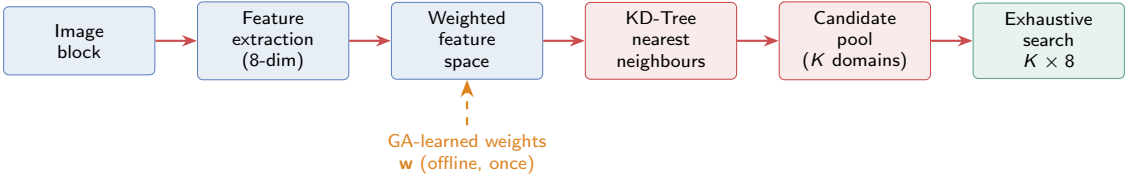
- **Isometry-LS**: try the other 7 isometries of the current domain.
- **Spatial-LS**: try the 8 spatially adjacent domain blocks.

*Lesson:* local search reliably raises PSNR — but adds time.

## 3. Feature-Guided PSO (FG-PSO)

First build a small feature-based candidate pool (top-$K$ domains per range block), then run Memetic PSO *inside* that pool. *Lesson:* shrinking the space to high-quality candidates helps much more than tuning the optimizer.

# FGDS pipeline: select first, then search




## Three online stages

1. Describe every block by 8 numbers.
2. Use a KD-Tree to find the $K$ most similar domains per range block.
3. Exhaustively try those $K \times 8$ candidates; keep the best.

## Why this is fast *and* good

- Search space per block: 520,200 $\rightarrow$ 640 ($K$=80).
- Exhaustive within the pool $\Rightarrow$ optimal, deterministic.
- Selection is structure-aware $\Rightarrow$ the pool already contains good matches.

# Stage 1: an 8-dimensional isometry-invariant descriptor

Each block $b$ ($L \times L$) becomes a vector:

$$\mathbf{f}(b) = [\ \underbrace{\mu, \sigma}_{\text{intensity}}\ ,\ \underbrace{\bar{g}_h, \bar{g}_v}_{\text{texture}},\ \underbrace{\hat{q}_1, \hat{q}_2, \hat{q}_3, \hat{q}_4}_{\text{layout (sorted)}}\ ]$$

- $\mu, \sigma$: mean & standard deviation.
- $\bar{g}_h, \bar{g}_v$: mean horizontal/vertical gradient magnitude (texture).
- $\hat{q}_{1..4}$: the four quadrant means, sorted ascending.

## Two requirements

**Discriminative** (separates different blocks)
**Isometry-invariant**: a domain under any rotation/flip is the *same* candidate (the isometry is searched later, in Stage 3).

**Why sorting the quadrants gives invariance**

| | |
|---|---|
| 3 | 7 |
| 1 | 5 |

original

| | |
|---|---|
| 1 | 3 |
| 5 | 7 |

rotated 90°

Quadrant values are *permuted* by rotation, but

$$\text{sort}(3, 7, 1, 5) = \text{sort}(1, 3, 5, 7) = (1, 3, 5, 7)$$

same vector $\Rightarrow$ fully invariant.

Features are $z$-score normalized so each dimension is comparable.

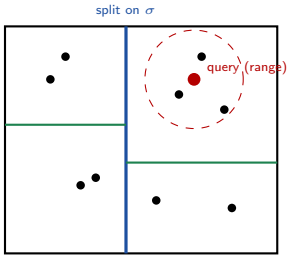
# What is a KD-Tree?

A KD-Tree is a binary tree that recursively splits points along alternating coordinate axes. It answers "**which points are nearest to this query?**" in $O(\log P)$ instead of $O(P)$.

- Split the data by the $x$-median, then each half by the $y$-median, and so on.
- To find neighbours, descend to the query's leaf and prune branches that are provably too far.
- Efficient in low dimensions (our space is only 8-D; trouble starts above $\sim$20).

## In our project

Points = the 8-D feature vectors of all $P$ **domain** blocks. Query = a **range** block's feature vector. Answer = its $K$ nearest domains.




The dashed circle is the $K$-nearest region; the tree lets us find it without scanning all points.

# How we use the KD-Tree on domain features

**The matching distance is weighted Euclidean:**

$$\mathcal{P}_i = \arg\min_{|S|=K} \sum_{j \in S} \left\| \mathbf{w} \odot (\hat{\mathbf{f}}_{R,i} - \hat{\mathbf{f}}_{D,j}) \right\|^2$$

**Trick:** pre-multiply *both* range and domain features by $\mathbf{w}$. Then weighted distance = plain Euclidean distance $\Rightarrow$ a standard KD-Tree works directly.

1. Build one KD-Tree from all $P$ weighted domain features.
2. Batch-query all $M$ range features at once for their top-$K$.

## The speedup that motivated this

Pairwise (brute-force) pool construction:

$$O(M \cdot P \cdot F) \approx \textbf{180 s}$$

KD-Tree pool construction:

$$O(PF \log P + MK \log P) < \textbf{0.5 s}$$

$\approx 1390\times$ faster, identical result.

## Important

The KD-Tree returns the **same** neighbours as brute force — so PSNR is *unchanged*. Only the time changes.

# Stage 3: exhaustive search inside the pool

For each range block, we now have only $K$ candidate domains. We try **all** $K \times 8$ (domain, isometry) pairs and keep the one with minimum MSE (subject to $|s| < 1$).

- With $K = 80$: 640 FFEs per block.
- This is *fewer* than the budget given to the PSO baselines.
- Yet it is **exact** within the pool and fully **deterministic** — no random restarts, no tuning.

## Why not PSO here?

In a 640-candidate space, PSO would (a) round continuous positions to indices and possibly skip candidates, and (b) need multiple runs for stability. A complete scan is simply better — this is the ablation that let us drop the optimizer.

## Cost reduction vs. full search

$3.4 \times 10^{10} \rightarrow 4.2 \times 10^{7}$ FFEs/image ($> 800\times$ fewer).

# Why a Genetic Algorithm? (and where it sits)

Equal weighting assumes all 8 features matter equally. **They do not** — recall: a mean mismatch is free, a contrast mismatch is not.

So we learn the weight vector $\mathbf{w} \in \mathbb{R}^8_{>0}$ that makes the candidate pool as good as possible.

## A different use of a metaheuristic

We *removed* PSO from block matching. The GA is used one level **up**: to **design the feature space**, not to match blocks.

- Runs **once, offline**, on a single training image.
- Costs **zero** at encoding time (just an element-wise multiply).
- Learned weights are reused for all images.

## What the GA optimizes

**Individual** = a weight vector $\mathbf{w}$ (8 genes).

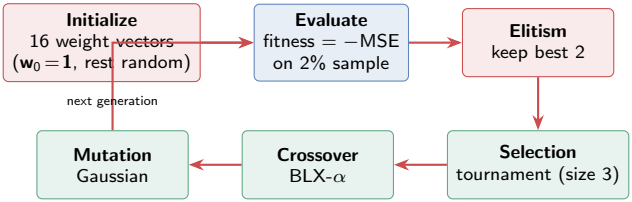
**Fitness** = average MSE on a 2% random sample of range blocks, after running *weighted features* → *KD-Tree* → *exhaustive search*.

Lower MSE = better. We maximize −MSE.

## Setup

Population 16, 10 generations, sample ratio 2%, $K = 80$, training image 0003 (**not** in the test set).

# GA workflow in our project




## Selection

Pick 3 individuals at random; the one with best fitness becomes a parent. Repeat for the second parent.

## Crossover (rate 0.8)

BLX-$\alpha$ blends two parents per gene (next slide).

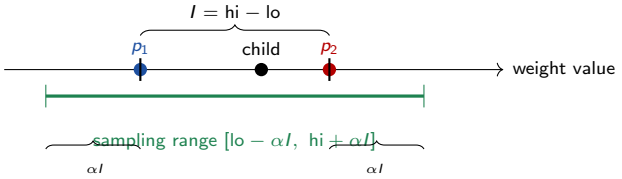
## Mutation (rate 0.4)

Perturb 1–3 genes by $\mathcal{N}(0, 0.25)$; clamp to be positive.

# How we get a new solution: BLX-$\alpha$ crossover

For each gene $d$, given parents $p_1, p_2$, let $\text{lo} = \min(p_1, p_2)$, $\text{hi} = \max(p_1, p2)$, $I = \text{hi} - \text{lo}$. The child gene is drawn uniformly from

$$\big[\,\text{lo} - \alpha I,\ \ \text{hi} + \alpha I\,\big], \quad \alpha = 0.5.$$




One gene shown. The child is a random point in the green range; repeating for all 8 genes produces one offspring weight vector.

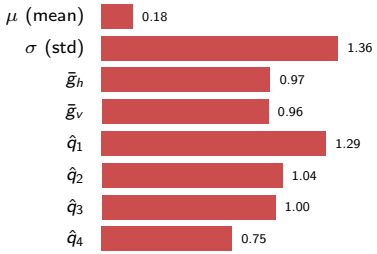
- Children can lie *between* parents **or slightly beyond** them (the $\alpha I$ margin) — this gives exploration.
- Applied gene-by-gene, so each of the 8 weights is blended independently.

## Then mutation

With prob. 0.4, add Gaussian noise to 1–3 random genes, then clamp $\geq 0.05$. This is how a genuinely **new** weight vector appears.

# What the GA learned (and why it generalizes)

Converged weights (train image 0003, sampled MSE 72.82 → 71.77):




## The result matches theory exactly

- $\mu$ smallest (0.18): a mean mismatch is absorbed by offset $o$ — so it should barely count.
- $\sigma$ largest (1.36): a contrast mismatch cannot be absorbed — so it should count most.
- Ratio $\sigma/\mu \approx \mathbf{7.6}\times$.

## Why one training image is enough

"$\sigma$ drives the multiplicative contrast, $\mu$ is free" is a property of the **affine model** (the formula for $s, o$), *not* of any image. So the weights transfer — and indeed they improve PSNR on **all 10** unseen test images.

# Experimental setup and the four experiments

## Setup

- 10 DIV2K images, $1024 \times 1024$, grayscale.
- $L=4$, domain 8, stride 4; CR $\approx 3.7{:}1$.
- FFE budget 800 per block (fair common axis).
- PSO methods: 5 runs each; FGDS deterministic.
- Significance: Wilcoxon signed-rank, $n=10$.
- GA trained on 0003 (held out).

## Four connected experiments

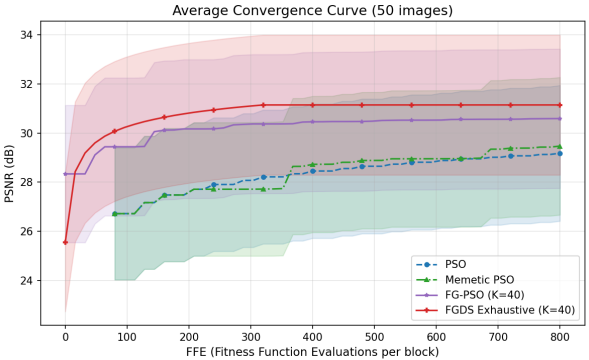
- **E1** PSO / Memetic / FG-PSO / FGDS ($K=40$) — shows the optimizer is unnecessary.
- **E2** FGDS pairwise / KD-Tree / KD-Tree+GA ($K=80$) — isolates the speedup and the GA gain.
- **E3** $K \in \{5, .., 160\}$ sweep — finds the sweet spot.
- **E4** feature pool vs. *random* pool ($K=80$) — proves the features matter.

# E1: the optimizer is not the bottleneck

| Method | PSNR (dB) | Time (s) |
|---|---|---|
| PSO | 28.87 | 1419.8 |
| Memetic PSO | 29.15 | 1317.2 |
| FG-PSO ($K$=40) | 30.27 | 1268.8 |
| **FGDS ($K$=40)** | **30.80** | **358.0** |

## Read it as a trail

LS helps (+0.28); feature pool helps more (+1.12); dropping PSO for exhaustive helps again (+0.53) — with $\sim$**4**$\times$ **speedup** and no randomness.




Feature-guided methods start 2–3 dB higher; FGDS saturates at $K \times 8 = 320$ FFEs and stays on top.

# E2: KD-Tree gives the same quality, far faster (+ GA)

| Variant | PSNR | CandSel(s) | MSE |
|---|---|---|---|
| Pairwise | 31.21 | 179.57 | 56.27 |
| KD-Tree | 31.21 | **0.13** | 56.27 |
| KD-Tree + GA | **31.26** | 0.12 | **55.60** |

## Two clean results

- KD-Tree = pairwise in quality (identical neighbours), but candidate selection drops 179.6 s → 0.13 s (∼1390×).
- GA weights add +0.05 dB on average, **positive on all 10 images**, at zero encoding cost.

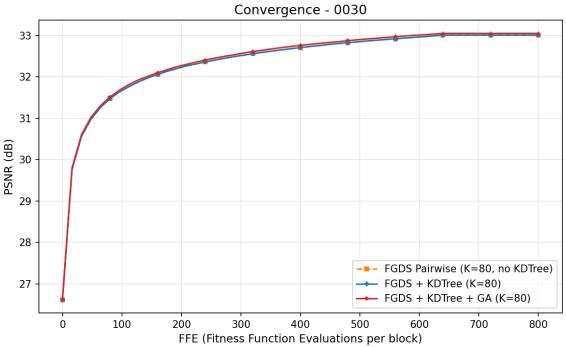



Image 0030: the three variants overlap almost perfectly (same neighbours); GA is marginally on top.

# E3: choosing the pool size $K$

| $K$ | FFE | PSNR (dB) | Total (s) |
|---|---|---|---|
| 5 | 40 | 29.18 | 51.6 |
| 10 | 80 | 29.80 | 95.7 |
| 20 | 160 | 30.33 | 183.6 |
| 40 | 320 | 30.80 | 360.9 |
| 80 | 640 | 31.21 | 712.9 |
| 160 | 1280 | 31.33 | 884.7 |

## Diminishing returns

- PSNR climbs steadily up to $K = 80$ (31.21 dB).
- $K = 80 \rightarrow 160$: only **+0.12 dB** for $\sim$1.24$\times$ the time.
- $\Rightarrow$ the top-80 feature neighbours already contain essentially all useful domains.

## Decision

Adopt $K = 80$ as the sweet spot (used in E2 and E4). Even $K = 40$ already beats every PSO baseline at a fraction of the time.

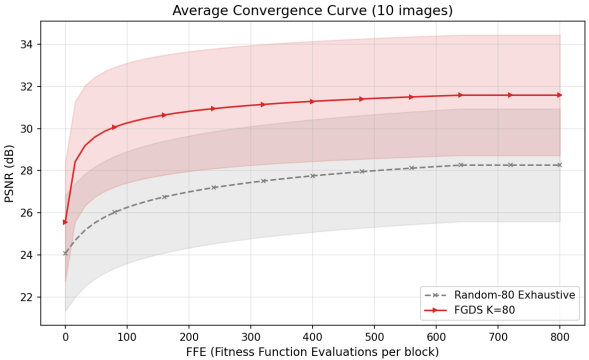
# E4: is it the features, or just a smaller pool?

**Control experiment.** Replace the feature-guided pool with a random pool of the *same* size ($K = 80$), same exhaustive search, same FFEs. Only the *selection* differs.

## Result

- FGDS-80: **31.21 dB** vs. Random-80: 27.96 dB
- Gap = **+3.25 dB**, FGDS wins on **all 10** images.

A same-size random pool is far worse ⇒ the quality comes from the **feature-guided selection**, not from shrinking the space alone. *This is the core of our thesis.*




At equal FFEs the feature-guided pool (red) dominates the random pool (gray) throughout by ∼3.25 dB.

# Statistical significance and a visual example

**Wilcoxon signed-rank (PSNR, $n = 10$)**

| Comparison | ΔPSNR | $p$ |
|---|---|---|
| FGDS vs. PSO | +1.93 | $< 0.01$ |
| FGDS vs. Memetic PSO | +1.65 | $< 0.01$ |
| FGDS vs. FG-PSO | +0.53 | $< 0.01$ |
| FGDS+GA vs. FGDS | +0.05 | $< 0.01$ |
| FGDS-80 vs. Random-80 | +3.25 | $< 0.01$ |

FGDS wins every paired image in every comparison ($W^+ = 55$, exact $p = 1/2^{10} \approx 0.001$).

Original (0030)

FGDS+GA, 32.73 dB

Ridges, snow boundaries, and sky gradients are preserved at CR $\approx$ 3.7:1.

# Conclusion

## What we showed

- For high-resolution FIC, the bottleneck is the **search space**, not the optimizer. A feature pool of 80 domains, searched exhaustively, beats PSO over $5 \times 10^5$ candidates by $\sim$1.9 dB.
- Once the pool is small, **exhaustive search dominates PSO** — so we remove the metaheuristic matcher entirely.
- A **KD-Tree** builds the pool $\sim$1390$\times$ faster than pairwise, with identical quality.
- A **GA** learns, offline and once, how much each feature matters; the learned weights ($\sigma \gg \mu$) follow from the affine model and transfer to all images.

## Our contribution

We move the heuristic from block **matching** to feature-guided block **selection**. Result: **+1.7–3.2 dB** over PSO, $\sim$**4**$\times$ **faster**, all gains significant at $p < 0.01$; a random-pool control attributes **+3.25 dB** specifically to the feature-guided selection.

# Thank you

## Feature-Guided Domain Selection
for Fast Fractal Image Compression

Zhen-Rong Wu  Darrin Lin  Liang-Yun Ko

National Taiwan Normal University

Source code: `github.com/noyapoyo/pso-ma-fic`

**Take-away**

When a metaheuristic struggles on a combinatorial problem, apply it to the *design of the search space* — not to the search itself.